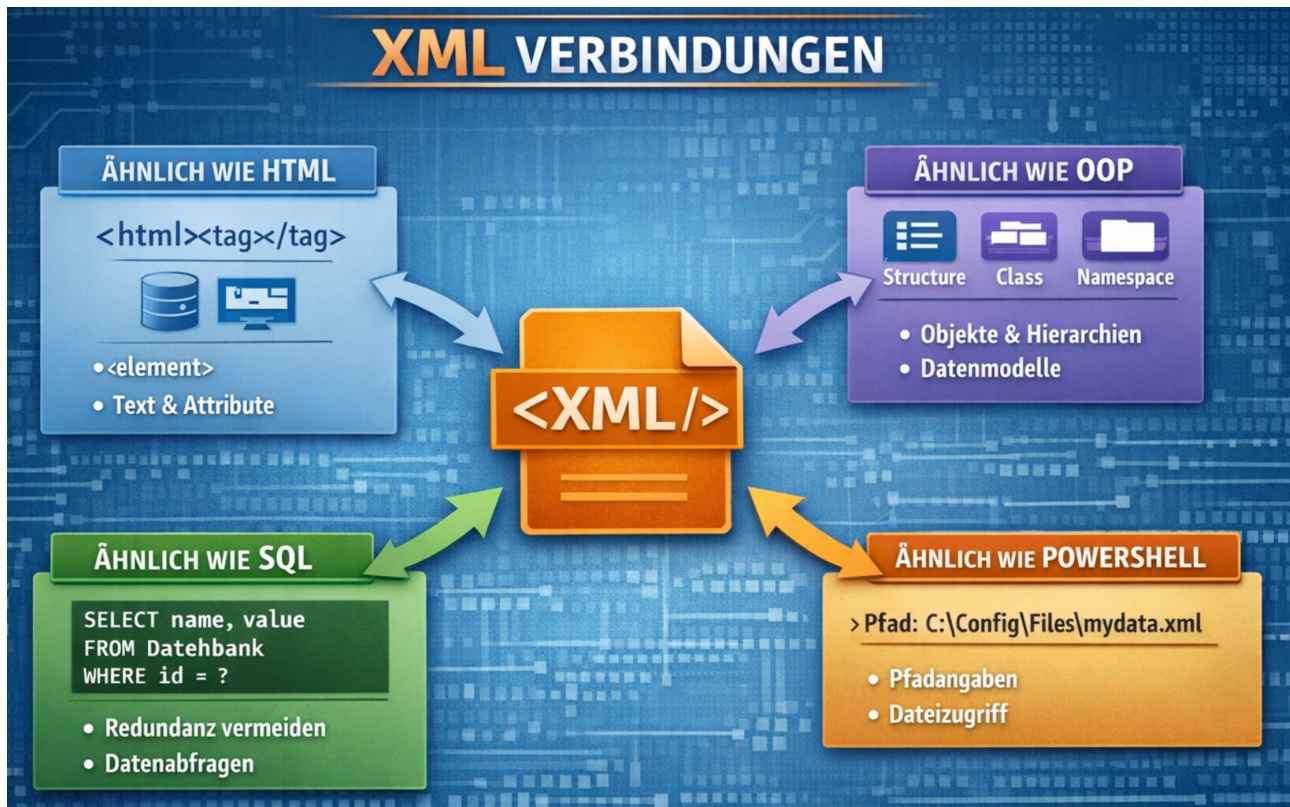


Kognitive Brücken - Analogiebildung



dieses Bild habe ich Ki erstellen lassen, auf mein Wissensstand.

XML + XPath Start

steht für Extensible Markup Language (Erweiterbare Auszeichnungssprache)
ist eine Auszeichnungssprache, ähnlich wie HTML, trennt aber Daten von der Darstellung
wurde entwickelt, um Daten zu speichern und zu transportieren
vereinfacht Datenaustausch, Datentransport, Plattform-Änderungen und Datenverfügbarkeit
bewirkt selbst keine Verarbeitung, sondern speichert Informationen hierarchisch in selbstdefinierten Tags im Klartextformat
XML-Tags sind komplett selbstdefiniert und können an praktisch jedes Speichermedium oder jede Datenstruktur angepasst werden

XML bewirkt selbst keine Verarbeitung, sondern speichert Informationen hierarchisch in selbst definierten Tags, im Klartextformat.

XML-Tags sind komplett selbstdefiniert und können an praktisch jedes Speichermedium oder jede Datenstruktur angepasst werden.

z.B.

XML Regeln: wohlgeformt

- XML-Deklaration: `<?xml version="1.0" encoding="UTF-8"?>` Standard, besser Dokumentation
braucht immer ein eindeutiges Root Element
- Alle Tags müssen geschlossen werden
- Die Verschachtelung und Groß u. Kleinschreibung muss eingehalten werden/ case-sensitive
- Sonderzeichen müssen escaped werden
- Leere Elemente sind selbstschließend, `<Element />`
- Elemente enthalten Daten, Aktiv– Attribute beschreiben Daten, Passiv

Baumstruktur:

```
<?xml version="1.0" encoding="UTF-8"?>
<kunden>
  <kunde>
    <name>Max Müller</name>
    <email>max@example.com</email>
  </kunde>
  <kunde>
    <name>Anna Schmidt</name>
    <email>anna@example.com</email>
  </kunde>
</kunden>
```

- XML-Deklaration
- Root-Element-Start
- Element-Start
- Tag – Wert – Tag
- Tag – Wert – Tag
- Element-Ende
- Element-Start
- Tag – Wert – Tag
- Tag – Wert – Tag
- Element-Ende
- Root-Element-Ende

SQL - Ausgabe in XML

```
SELECT
  CONCAT
    ( '<kunden>\n',
      GROUP_CONCAT(
        CONCAT(
          '<kunde>',
            '<name>', name, '</name>',
            '<email>', email, '</email>',
            '</kunde>'
        ) SEPARATOR '\n'
      ), '\n</kunden>'
    ) AS xml_output
FROM kunden;
```

- Sucht Daten
- Verbindet Daten
- Root-Element-Start
- Gruppierter Verkettung
- Verbindet Daten
- Element-Start
- Tag – Wert – Tag
- Tag – Wert – Tag
- Element-Ende
- Trennzeichen
- Root-Element-Ende
- Spaltenbenennung
- Von Tabelle

Elemente / Tags mit Attribut

- Attributswerte müssen in "wert" Anführungszeichen stehen,
- hat es selbst Anführungszeichen ' "wert" ' oder "wert";
- Attribute dürfen nicht mehrere Werte enthalten
- Attribute dürfen keine Baumstrukturen enthalten
- Die Attribute sind nicht einfach erweiterbar
- CSS erkennt class und id Attribute in XML, wie in HTML

```
<bookstore>
  <book category="cooking">
    <title lang="en">Everyday Italian</title>
  </book>
</bookstore>
```

- Root-Start
- child-Start
- subchild
- child-Ende
- Root-Ende

Elemente mit id-Attribut

- Nutzung für Referenzierung in XML/HTML, CSS-Styling, JavaScript, Verlinkung

Datenbanknutzung:

zb.

```
<note id="501">
```

```

        <titel>Einkaufsliste</titel>
        <text>Milch, Brot, Käse</text>
        <Preis>9.99</Preis>
    </note>

```

Datenbank import

selbst schreiben als Beispiel in js

//Variable definieren//

```
DB_Data=mapNote(xmlNote)
```

//Mapping + Validierung//

```
function mapNote(xmlNote) {
```

```

    const id = Number(xmlNote.getAttribute('id'));
    if (!Number.isInteger(id)) {
        throw new Error('ID muss Integer sein');
    }

```

- Funktionsstart

- id constante erstellen
- IF-Abfrage(ist nicht Zahl?)
- Fehlerausgabe

```

    const titel = xmlNote.querySelector('titel')?.textContent?.trim();
    if (!titel) {
        throw new Error('Titel fehlt');
    }

```

- titel constante erstellen
- IF-Abfrage (Wert in Titel?)
- Fehlerausgabe

```
const text = xmlNote.querySelector('text')?.textContent?.trim() ?? null;
```

- prüfe steht was im Text?

```

    const preisRaw = xmlNote.querySelector('Preis')?.textContent;
    let preis = null;
    if (preisRaw !== undefined) {
        const trimmed = preisRaw.trim();
        if (trimmed === "") {
            throw new Error("Preis ist leer");
        }
        const parsed = Number(trimmed);
        if (!Number.isFinite(parsed)) {
            throw new Error("Preis ist keine gültige Zahl");
        }
        preis = parsed;
    }

```

- preisRaw constante erstellen
- temporäre preis variable
- preisRaw ist definiert
- trimmed constante erstellen
- Prüfung trimmed = leer
- Fehlerausgabe

- IF-Abfrag(Wert zu groß,klein)
- Fehlerausgabe

- preis übernahme

```
return { id, titel, text, preis };
```

- Datenrückgabe
- Funktionsende

//DB Eintrag//

```
const stmt = db.prepare(` INSERT INTO notizen (id, titel, text, preis) VALUES (?, ?, ?, ?) `);
```

- DB Eintrag Vorbereitung

```
for (const note of DB_Data)
```

- Schleifenstart

```

    { stmt.run(note.id, note.titel, note.text, note.preis);
    }

```

- Daten in query einsetzen

```
stmt.finalize();
```

- DB Eintrag

mehrere gleiche Attribute oder Elemente müssen durch eine for oder forEach Schleife.

zB.

```
const allNotes = document.querySelectorAll('note');
```

- Alle <note> finden

```
allNotes.forEach(note => {
    const noteld = note.getAttribute('id');
    const titel = note.querySelector('titel').textContent;
    db.prepare('INSERT INTO notes (id, titel) VALUES (?, ?)').run(noteld, titel);
    console.log(`Gespeichert: ID ${noteld} - ${titel}`);
});
```

- Schleifenstart
- id speichern
- titel speichern
- in DB schreiben, InjecSchutz
- Consolenausgabe
- Schleifenende

oder

```
const allNotes = document.querySelectorAll('note');
```

- Alle <note> finden

```
const DB_Data = [];
allNotes.forEach(xmlNote => {
    DB_Data.push(mapNote(xmlNote));
});
```

- DB_Data Array erstellt
- Schleifenstart
- Schleifendurchlauf
- Schleifenende

```
for (const note of DB_Data) {
    stmt.run(note.id, note.titel, note.text, note.preis);
}
stmt.finalize();
```

- Schleife
- Daten in query einsetzen
- Datenübertragung

Performance kritisch

- große Datenmengen verarbeitet (ETL, Imports, BI)
- komplexe JOINS, Window Functions, CTEs
- Injection-Schutz selbst schreiben
- Kritisch:
 - IDs / Foreign Keys
 - Zahlen mit Bedeutung (Mengen, Preise, Limits)
 - Enums / Statuswerte
 - Datum / Zeit
 - Sicherheitsrelevante Felder
- Nicht kritisch:
 - Freitext (titel, beschreibung)
 - Kommentare
 - optionale Notizen

ORM (Object-Relational Mapping)

```
await Note.create({
    id: noteld,
    titel: titel
});
```

CRUD(Create, Read, Update, Delete) -lastige Anwendungen

- Datenmodell stabil
- Tabellen bleiben bestehen
- Spalten ändern sich selten
- Beziehungen sind klar
- Geschäftsregeln ändern sich nicht dauernd
- Wartbarkeit wichtiger als Performance

- Automatischer Injection-Schutz

Xpath:

Merke!!! Im Arbeitsschritt debuggen, im Ergebnis debug entfernen. **!!!Merke**

XPath steht für XML-Pfadsprache, verwendet eine „pfadähnliche“ Syntax, um Knoten in einem XML-Dokument zu identifizieren und zu navigieren, ist ein wichtiger Bestandteil des XSLT-Standards.

Es sieben Arten von Knoten: Element-, Attribut-, Text-, Namensraum-, Verarbeitungsanweisungs-, Kommentar- und Rootknoten.

<bookstore>(root element node)

<author>J K. Rowling</author> (element node)

lang="en" (attribute node)

Der XML-Parser baut einen Baum.

Jeder Baustein in diesem Baum ist ein Knoten.

Die wichtigsten Knotentypen

1. Element-Knoten

<note>...</note>

Strukturträger

können Kinder haben

2. Attribut-Knoten

<note id="501">

hängen an Elementen

haben atomare Werte

3. Text-Knoten

<titel>Einkaufsliste</titel>

"Einkaufsliste" ist ein Text-Knoten

enthält einen atomaren Wert

4. Root-Knoten

oberster Knoten des Dokuments

meist ein Element

5. Weitere (der Vollständigkeit halber)

Kommentar-Knoten <!-- -->

Processing Instruction

Namespace-Knoten

Alle Pfadausdrücke können miteinander kombiniert werden.

Pfadausdrücke

/	-	Root-Auswahl
//	-	überall suchen
.	-	aktueller Knoten

..	-	Parent-Knoten
./*	-	Alle child
@	-	Attribute
[]	-	Filter
*	-	Filter
	-	UND-Verknüpfung

Alle XPath-Funktionen können miteinander kombiniert werden, solange der DatenTypen zueinander passt.

nodeName[not(Knoten)]	-	Schließt alles mit dem Knoten jeder Art aus
text()	-	gibt den Text-Knoten eines Knotens zurück
string()	-	wandelt Knoten-Text in String um
contains(nodeName, "WertNamen")	-	Ist diese wert in diesen Knoten
nodeName[starts-with(Element, "WertNamen")]	-	Beginnt ein Knoten mit demWertNamen
Count(//nodeName)	-	Anzahl der Knoten
nodeName[position() = 1]	-	Erstes Element
last()	-	Letztes Element

Standortpfadausdruck / Achsen

child::book	Wählt alle book-Knoten , die direkte Kinder des aktuellen Knotens sind.
attribute::lang	Wählt das Attribut lang des aktuellen Knotens.
child::*	Wählt alle Element-Kinder des aktuellen Knotens (egal wie sie heißen).
attribute::*	Wählt alle Attribute des aktuellen Knotens.
child::text()	Wählt alle Textknoten , die direkte Kinder des aktuellen Knotens sind.
child::node()	Wählt alle Knoten , die direkte Kinder des aktuellen Knotens sind. (Elemente, Text, Kommentare, etc.).
descendant::book	Wählt alle book-Knoten , die irgendwo unterhalb des aktuellen Knotens liegen (Kinder, Enkel, Urenkel ...).
ancestor::book	Wählt alle book-Knoten , die Vorfahren des aktuellen Knotens sind (Eltern, Großeltern ...).
ancestor-or-self::book	Wählt alle book-Vorfahren des aktuellen Knotens und zusätzlich den aktuellen Knoten selbst , falls dieser ein book-Knoten ist.

<code>child::* / child::price</code>	Wählt alle price-Knoten , die Enkel des aktuellen Knotens sind (also Kinder von Kindern des aktuellen Knotens).
--------------------------------------	---

XML + XPath Ende

XSD

XSD Start

XSD ist vom Denken her wie ein struct / class in anderen Sprachen, ohne Verhaltenmuster.

Datengültigkeit wird erstellt, XSD macht aus freiem XML eine regelbasierte Struktur.

**zB.
Python**

library.py

```
class Book:
    def __init__(self, title: str = "", price: float = 0.0):
        self.title = title
        self.price = price
```

Projekt Code:

```
from library import Book
```

```
my_book = Book("1984", 15.99)
```

oder

```
my_book = Book( )
my_book.title = "1984"
my_book.price = "15.99"
```

- Bibliotheksname

- Class name / Objekt-Name
- Datentyp und defaultwert
- festgelegte string Variable titel
- festgelegte dezimal Variable price

- Book von Bibliothek einbinden

- Objekt instanziiieren mit Werten

- Book-Objekt instanziiieren
- title Wert setzen
- price Wert setzen

XML

book-library.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:element name="book">
    <xs:complexType>
```

- Version
- Schema start / Root start
- Element/XML-schema-Name
- struc/class complex hat childs

```

        <xs:sequence>
            <xs:element name="title" type="xs:string"/>
            <xs:element name="price" type="xs:decimal" minOccurs="0"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
</xs:schema>

```

- setzt die Reihenfolge fest
- festgelegte string Variable titel
- festgelegte dezimal Variable price
- sequence ende
- class ende
- Element/Objekt ende
- Schema Ende / Root Ende

```

<?xml version="1.0" encoding="UTF-8"?>
<book xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:noNamespaceSchemaLocation="book-library.xsd">
    <title>1984</title>
    <price>15.99</price>
</book>

```

- Version
- Objekt instanziiieren
- Bibliothek einbinden
- title Wert setzen
- price Wert setzen
- Schema Ende / Root Ende

Grundstruktur:

```

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<!-- Hier kommen deine Definitionen -->
</xs:schema>

```

Einfache Elemente: standart genau 1 Wert setzen

```

<xs:element name="WERT" type="xs:Datentyp"/>

```

Datentyp:

xs:string - Text
 xs:integer - Ganzzahl
 xs:decimal - Dezimalzahl
 xs:boolean - true/false
 xs:date - Datum (YYYY-MM-DD)
 xs:time - Zeit (HH:MM:SS)
 xs:dateTime - Datum + Zeit
 xs:anyURI - URL/URI

Komplexe Elemente:

```

<xs:element name="person">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="WERT" type="xs:Datentyp"/>
            <xs:element name="WERT" type="xs:Datentyp"/>
            <xs:element name="WERT" type="xs:Datentyp"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>

```

Häufigkeit/Vorkommen:

Beliebig viele Werte:

maxOccurs="unbounded" (Liste/Array)

```

<xs:element name="phone" type="xs:string" maxOccurs="unbounded"/>

```

```
<phone>111</phone>
<phone>222</phone>
<phone>333</phone>
....
```

Optional: Kann Wert entgegen nehmen, braucht aber nicht

```
<xs:element name="phone" type="xs:string" minOccurs="0"/>

<!-- kein Phone -->

oder

<phone>01215487521</phone>
```

Begrenzte Liste/Array - minOccurs="1" maxOccurs="5" = maxOccurs="5"

```
<phone>111</phone>
<phone>222</phone>
<phone>333</phone>
```

Optionale Liste: Kann Wert entgegen nehmen, braucht aber nicht, minOccurs="0" maxOccurs="2"

```
<xs:element name="phone" type="xs:string" minOccurs="0" maxOccurs="2"/>

<!-- kein Phone -->

oder

<phone>111</phone>
<phone>222</phone>
```

Attribute:

```
name="" type="" use="optional"
name="" type=""
```

- kann Attribut-Wert festlegen, muss nicht
- muss Attribut-Wert festlegen

Element-Attribute und Child-Werte:

XSD

```
<xs:element name="person">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="child1" type="xs:string"/>
      <xs:element name="child2" type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="attr1" type="xs:string"/>
    <xs:attribute name="attr2" type="xs:string"/>
  </xs:complexType>
</xs:element>
```

XML

```
<person attr1="" attr2="">
  <child1> Person1 </child1>
  <child2> Geburtstag </child2>
</person>
```

Element-Attribute und Wert:

XSD

```
<xs:element name="title">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="language" type="xs:string"/>
        <xs:attribute name="aktiv" type="xs:boolean"/>
        <xs:attribute name="rating" type="xs:decimal"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

XML

```
<title language="" aktiv="" rating=""> Buchtitel </title>
```

Typ-Erstellung:

XSD Salami Slice:

ref = Pointer/Referenz auf einen Speicherplatz

name = Neue Variable direkt hier erstellen

Um Element-Attribute und Wert nicht immer wiederholen zu müssen, Redundanzen vermeiden, Typ-Erstellung. Globalelement keine Childs und es wird darauf Referenziert. Bessere Wartbarkeit.

```
<xs:schema>
  <xs:complexType name="StringMitAttrs">
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="attr1" type="xs:string"/>
        <xs:attribute name="attr2" type="xs:string"/>
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>

  <xs:element name="c1" type="StringMitAttrs"/>
  <xs:element name="c2" type="StringMitAttrs"/>

  <xs:complexType name="PersonType">
    <xs:sequence>
      <xs:element ref="c1"/>
      <xs:element ref="c2"/>
    </xs:sequence>
  </xs:complexType>

  <xs:element name="person" type="PersonType"/>
</xs:schema>
```

lokal: Mischschema

Typ-Nutzung: alle Childs hat alle Attribute

```
<xs:element name="person">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="c1" type="StringMitAttrs" use="required"/>
      <xs:element name="c2" type="StringMitAttrs" use="optional"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
</xs:schema>
```

XML

```
<person>
  <child1 attr1="" attr2="">Max Muster</child1>
  <child2 attr1="" attr2="">MaxMuster@muster.de</child2>
</person>
```

xs:restriction

- definiert einen Value-Typ (Wertelogik).

xs:extension

- verwendet diesen Value-Typ als Base, um Attribute hinzuzufügen.

String Typen	
length	<xs:length value="5"/> - Exakte Länge
minLength / maxLength	<xs:minLength value="5"/> <xs:maxLength value="10"/> - Längenbereich
pattern	<xs:pattern value="[A-Za-z0-9]"/> - Regex
enumeration	<xs:enumeration value="positiv"/> <xs:enumeration value="negativ"/> <xs:enumeration value="normal"/> - whitelist
whiteSpace	preserve / replace / collapse

Numerische Typen	
minInclusive / maxInclusive	<xs:minInclusive value="0"/> <xs:maxInclusive value="100"/> - inklusiver Bereich
minExclusive / maxExclusive	<xs:minExclusive value="0"/> <xs:maxExclusive value="100"/> - exklusiver Bereich größer 0 kleiner 100
totalDigits	<xs:totalDigits value="5"/> - max. Ziffern
fractionDigits	<xs:fractionDigits value="2"/> - Nachkommastellen
enumeration	<xs:enumeration value="10"/> <xs:enumeration value="20"/> <xs:enumeration value="50"/> - feste Zahlenauswahl

Datum / Zeit (xs:date, xs:dateTime, xs:time)	
minInclusive / maxInclusive	<xs:minInclusive value="2025-01-01"/> <xs:maxInclusive value="2026-12-31"/> - Zeitbereich
pattern	<xs:pattern value="\d{4}-\d{2}-\d{2}"/> - Format erzwingen
enumeration	<xs:enumeration value="2026-01-01"/> <xs:enumeration value="2026-12-31"/> feste Zeitpunktauswahl

Boolean	
enumeration	<xs:enumeration value="0"/> - <xs:enumeration value="true"/> <xs:enumeration value="1"/> - <xs:enumeration value="false"/>

Merke!!! Man erstellt zuerst einen ValueType mit restriction und nutzt diesen anschließend als extension base. **!!!Merke**

was darf ein Zeichen sein

[A-Za-z0-9_] - Alle Zeichen in der [] in Kombi

wie viele Zeichen insgesamt

[] - min 1

[]+ - min 1 bis beliebig viele

* - 0 bis beliebig viele

{ }

- begrenzend auf, (genau - {5}), (min. bis - {5,10}), (min. {5,})

```
<xs:simpleType name="IdValue">
  <xs:restriction base="xs:string">
    <xs:pattern value="[A-Z]{3}-[0-9]{6}"/>
  </xs:restriction>
</xs:simpleType>
```

```
<xs:complexType name="IdType">
  <xs:simpleContent>
    <xs:extension base="IdValue">
      <xs:attribute name="system" type="xs:string" use="required"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
```

XSD Namespace:

xmlns:xs="http://www.w3.org/2001/XMLSchema"

- Bindet das xs: an das XML Schema

- Das xs kann auch anders heißen, zb. xmlns:ab, xs ist aber Standard

targetNamespace="urn:company:process"

- Definiert den Namespace des Dokuments

xmlns="urn:company:process"

- Setzt den Default-Namespace im XSD

elementFormDefault="qualified"

- Alle Element stehen im Namespace

XSD

Namespace-Entry-Point

Der Entry-Point nutzt include, um alle Teil-Schemas dieses Namespaces zusammenzuführen. Dadurch können Dateien im Namespace alle types untereinander nutzen.

z.B.

Dateiname - process.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<xs:schema
```

```
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
```

```
  targetNamespace="urn:company:process"
```

```
  xmlns="urn:company:process"
```

```
  elementFormDefault="qualified">
```

```
    <xs:include schemaLocation="process_customer.xsd"/>
```

```
    <xs:include schemaLocation="process_member.xsd"/>
```

```
</xs:schema>
```

Dateiname - process_customer.xsd

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="urn:company:process"
  xmlns="urn:company:process"
  elementFormDefault="qualified">

  <xs:element name="order">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="id" type="xs:int"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<order xmlns="urn:company:process">
  <id>123</id>
</order>
```

include und import

Merke!!! Man importiert immer ganzen Namespace, deshalb gut Namespaces klein halten wenn möglich. **!!!Merke**

include - gleicher namespace

Zum zusammenfügen von z.B Namespace XSD's in eine **Namespace-Entry-Point XSD**.

import - anderer namespace

Gilt nur in der XSD in der sie gebraucht wird, erstellte type können von weiteren XSD genutzt werden ohne import dessen, kein doppeltes Importieren nötig, Abhängigkeiten sind gekapselt, Validator löst das intern.

```
<xs:import
  namespace="urn:company:common"
  schemaLocation="common.xsd"/>
```

XSD Ende

XSLT und XSL-FO Start

XSLT - transformiert XML in XML/HTML/Text (Logik & Umformung)

Transformiert Struktur in Werte.

process_order.XSLT

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet
  version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:p="urn:company:process">

  <xsl:output method="text" encoding="UTF-8"/>

  <xsl:template match="/">
    <result>
      <xsl:value-of select="/p:order/p:id"/>

    </result>
  </xsl:template>
</xsl:stylesheet>
```

-
- **Version**
- **xsl Namespace variable**
- **p eigene Namespace variable**
- **Startpunkt der Transformation**
- **eigenes Ergebniselement**
- **wie xpath nur immer mit dem namespace zusatz**
es erstellt immer einen string

Man kann Quasi alle strukturen für Formate bauen die an möchte und weiß wie sie aussehen müssen.
Es gilt für xsl:output ist es kein xlm oder html ist es text(CSV,JSON,SQL).

XSL-FO - beschreibt Layout meist PDF (Layout & Ausgabe)

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:template match="/">
  <fo:root xmlns:fo="http://www.w3.org/1999/XSL/Format">

    <fo:layout-master-set>
      <fo:simple-page-master
        master-name="A4"
        page-width="21cm"
        page-height="29.7cm"
        margin="2cm">
        <fo:region-body/>
      </fo:simple-page-master>
    </fo:layout-master-set>

    <fo:page-sequence master-reference="A4">
      <fo:flow flow-name="xsl-region-body">
        <fo:block>
          ID:
          <xsl:value-of select="/p:order/p:id"/>
        </fo:block>
      </fo:flow>
    </fo:page-sequence>

  </fo:root>
</xsl:template>
```